

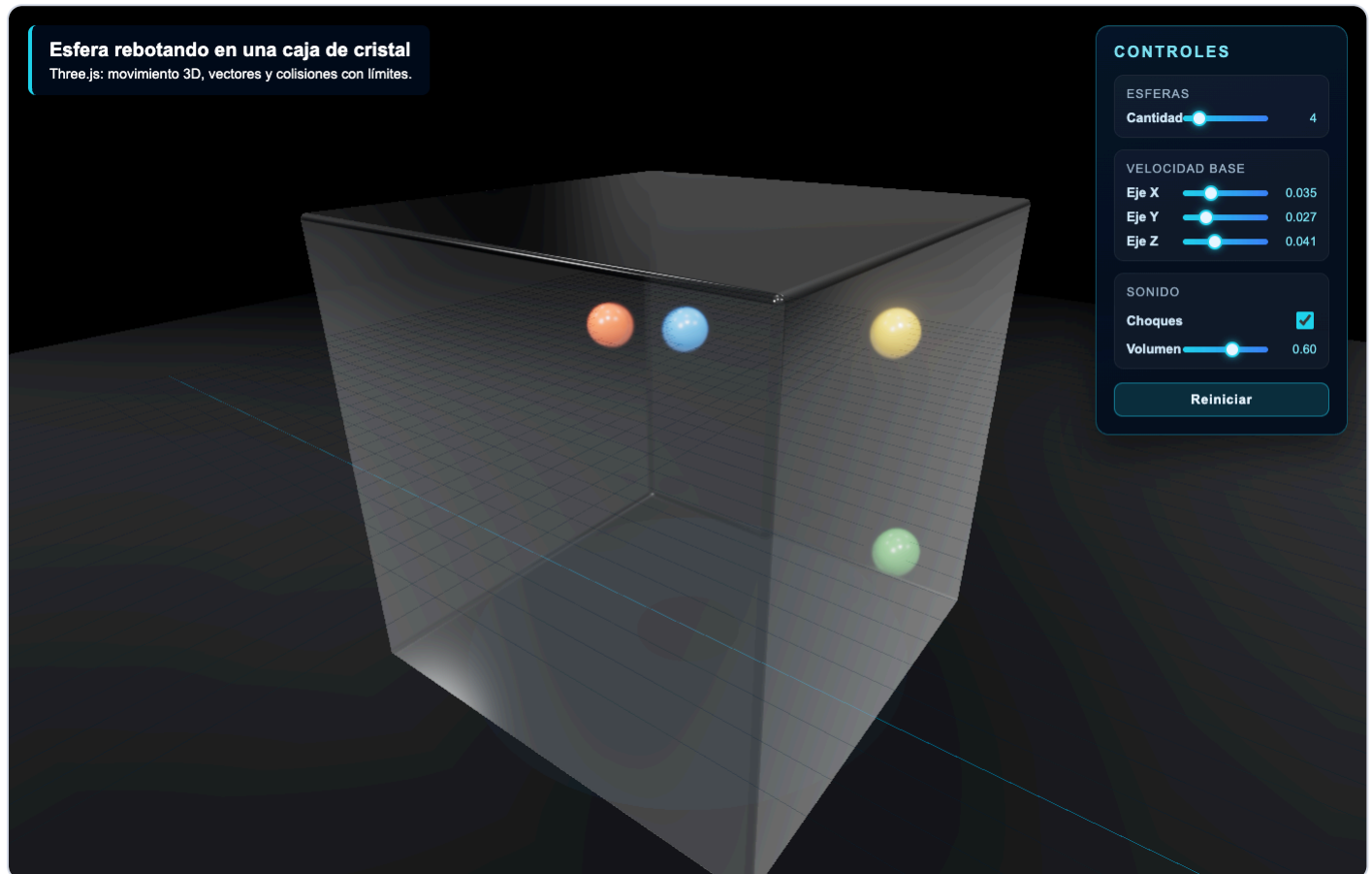
Esfera rebotando en una caja de cristal

Práctica de Realidad Virtual — Three.js: movimiento 3D, vectores, colisiones y render realista

Autor: Alonso · l24200197@pachuca.tecnm.mx

Fecha: 02/09/2026 · Repositorio: github.com/l24200197-commits/esfera_caja_threejs

Three.js r0.180.0 (vía importmap desde jsDelivr CDN)



Render final: cubo de cristal sin bordes, piso pulido con sombras, esferas tipo canica y bloom. Captura headless; en GPU real el cristal refracta con más detalle.

1. Descripción general

La aplicación simula una o varias esferas moviéndose en línea recta dentro de un cubo de cristal. Cada esfera guarda su posición y un vector de velocidad; en cada fotograma se integra la posición y se comprueban colisiones contra las seis caras del cubo y contra las demás esferas. Cada impacto genera una marca visual sobre la cara, una ráfaga de chispas, un sonido sintetizado y un pulso de luz en el piso. Un panel de controles permite ajustar en vivo el número de esferas, la velocidad base por eje y el sonido, botón de reinicio.

Características

- Movimiento 3D con vectores de velocidad e integración por fotograma.
- Colisiones esfera–cara (rebote por reflexión de la componente) y esfera–esfera (choque elástico de masas iguales).
- Realimentación de impacto: anillos, sistema de partículas (chispas), sonido (Web Audio API) y halo reactivo en el piso.
- Panel de controles: cantidad de esferas, velocidad base X/Y/Z, activación y volumen del sonido, botón de reinicio.
- Render realista: cristal con transmisión/refracción, entorno HDR, sombras suaves, niebla y post-proceso con *bloom*.
- Cámara orbital (`OrbitControls`) y adaptación a redimensionado de ventana.

2. Historial de versiones

v0.1	Estructura inicial de la práctica
v0.2	Cámara dinámica (OrbitControls)
v0.3	Marcas de impacto al tocar las caras
v0.4	Caja de controles con desplazadores de velocidad
v0.5	N esferas con colisiones entre sí, chispas y sonido
v0.6	Plano: piso espejo, rejilla luminosa, sombras y halo reactivo
v0.7	Cubo de cristal sin bordes y acabado realista

3. Arquitectura y módulos

Escena, cámara y renderer

main.js · 10–41

Se crea la escena con fondo oscuro y niebla (`THREE.Fog`) para dar profundidad. Cámara en perspectiva de 60° colocada en (9, 7, 12). El `WebGLRenderer` activa sombras suaves (`PCSoftShadowMap`) y tone mapping cinematográfico `ACESFilmicToneMapping` .

Entorno HDR

main.js · 32–35

`PMREMGGenerator` + `RoomEnvironment` generan un mapa de entorno HDR sintético que ilumina y se refleja en el cristal y las esferas, dándoles aspecto realista sin cargar imágenes externas.

Iluminación

main.js · 43–56

Luz ambiental + hemisférica (relleno frío/cálido) y una direccional que proyecta la sombra. La cámara de sombra se acota a la zona de la caja para maximizar la resolución del mapa (2048×2048).

Cubo de cristal

main.js · 58–85

`RoundedBoxGeometry` (aristas redondeadas, sin wireframe). Material `MeshPhysicalMaterial` con `transmission: 1`, `ior: 1.48`, `thickness` y `clearcoat` : cristal transparente con refracción real. `limit = boxSize/2 - radius` es el plano de rebote de los centros.

Piso y rejilla

main.js · 87–112

Plano tipo obsidiana pulida (`MeshStandardMaterial` oscuro que recibe sombras) + `GridHelper` tenue que se desvanece con la niebla. Se descartó el espejo perfecto (`Reflector`) porque generaba destellos con el bloom.

Halo reactivo del piso

main.js · 114–133

Anillo aditivo bajo la caja. `pulseFloor(amount)` incrementa `glowPulse` en cada choque; en el bucle decae suavemente y modula opacidad, escala y el brillo de la rejilla. También "respira" con una senoidal.

Esferas

main.js · 135–188

`addSphere()` / `removeSphere()` / `setSphereCount(n)` administran el arreglo `spheres` (cada elemento: `{ mesh, velocity }`). Material tipo canica (`clearcoat`), color rotativo, proyecta y recibe sombra. Posición y signo de velocidad aleatorios.

Velocidad base

main.js · 143–198

`baseSpeed` guarda el módulo de velocidad por eje. `applyBaseSpeed()` reescala todas las esferas conservando el sentido de cada componente, para que mover un slider se note al instante.

Caja de controles

main.js · 200–265

Cantidad (1–24) llama a `setSphereCount`. **Eje X/Y/Z** actualizan `baseSpeed` y el texto. **Choques** y **Volumen** controlan el audio. **Reiniciar** restaura velocidades por defecto y recoloca las esferas.

Sonido de choque

main.js · 267–313

Web Audio API sin archivos: `ensureAudio()` crea/reanuda el `AudioContext` tras el primer clic (política del navegador). `playClack(intensity)` sintetiza un "clack" = ráfaga de ruido con decaimiento cúbico → filtro *bandpass* → ganancia. El tono y el volumen escalan con la fuerza del impacto; hay un límite de 25 ms entre sonidos.

Marcas de impacto

main.js · 315–348

`spawnMarker(position, normal)` coloca un anillo (`RingGeometry`) orientado con `lookAt` sobre la cara golpeada. `updateMarkers(delta)` lo expande y desvanece durante 0.7 s y libera su material.

Chispas

main.js · 350–408

`spawnSparks(position, intensity, tint)` crea un `THREE.Points` con partículas que salen en direcciones aleatorias. `updateSparks(delta)` integra su posición con rozamiento ($\times 0.9$) y baja la opacidad; vida ≈ 0.45 s. Tope de 40 ráfagas simultáneas.

Colisiones con las caras

main.js · 415–435

Para cada eje: si $|\text{pos}| \geq \text{limit}$ se invierte esa componente de la velocidad y se hace `clamp`. Se calcula el punto de contacto y la normal y se disparan marca + chispas + sonido + pulso del piso.

Colisiones entre esferas

main.js · 437–463

Para cada par: si la distancia $< 2 \cdot \text{radio}$, se separa el solapamiento a partes iguales y se aplica un **choque elástico de masas iguales** — se intercambia la componente de la velocidad a lo largo de la normal: $v \pm ((v_A - v_B) \cdot n) \cdot n$. Solo responde si se están acercando ($\text{sep} > 0$).

Post-proceso (bloom)

main.js · 465–475

`EffectComposer`: `RenderPass` → `UnrealBloomPass` (strength 0.32, radius 0.5, threshold 0.86) → `OutputPass`. El umbral alto hace que solo brillen las chispas, el halo y los reflejos más intensos.

Bucle de animación

main.js · 477–519

`setAnimationLoop` con `delta` real acotado. Orden por frame: mover esferas y colisionar con paredes → colisiones entre pares → actualizar marcas y chispas → halo del piso → `controls.update()` → `composer.render()`. El `resize` ajusta cámara, render, composer y bloom.

4. Código fuente documentado

4.1 index.html

Estructura de la página, importmap de Three.js y marcado de la caja de controles (sliders, casilla de sonido y botón de reinicio).

index.html

```
1  <!DOCTYPE html>
2  <html lang="es">
3
4  <head>
5    <meta charset="UTF-8">
6    <meta name="viewport" content="width=device-width, initial-scale=1.0">
7    <title>Práctica 1.2 | Esfera rebotando en una caja de cristal</title>
8    <link rel="stylesheet" href="./assets/css/styles.css">
9
10   <script type="importmap">
11     {
12       "imports": {
13         "three": "https://cdn.jsdelivr.net/npm/three@0.180.0/build/three.module.js",
14         "three/addons/": "https://cdn.jsdelivr.net/npm/three@0.180.0/examples/jsm/"
15       }
16     }
17   </script>
18 </head>
19
20 <body>
21   <header class="panel">
22     <h1>Esfera rebotando en una caja de cristal</h1>
23     <p>Three.js: movimiento 3D, vectores y colisiones con límites.</p>
24   </header>
25
26   <aside class="controls" id="controls">
27     <h2>Controles</h2>
28
29     <div class="control-group">
30       <span class="control-group__title">Esferas</span>
31
32       <label class="control" for="count">
33         <span class="control__name">Cantidad</span>
34         <input type="range" id="count" min="1" max="24" step="1" value="4">
35         <output class="control__value" id="countOut">4</output>
36       </label>
37     </div>
38
39     <div class="control-group">
40       <span class="control-group__title">Velocidad base</span>
41
42       <label class="control" for="velX">
43         <span class="control__name">Eje X</span>
44         <input type="range" id="velX" min="0" max="0.12" step="0.001" value="0.035">
45         <output class="control__value" id="velXOut">0.035</output>
46       </label>
47
48       <label class="control" for="velY">
49         <span class="control__name">Eje Y</span>
50         <input type="range" id="velY" min="0" max="0.12" step="0.001" value="0.027">
51         <output class="control__value" id="velYOut">0.027</output>
52       </label>
53
54       <label class="control" for="velZ">
55         <span class="control__name">Eje Z</span>
56         <input type="range" id="velZ" min="0" max="0.12" step="0.001" value="0.041">
57         <output class="control__value" id="velZOut">0.041</output>
58       </label>
59     </div>
60
```

```

61     <div class="control-group">
62         <span class="control-group__title">Sonido</span>
63
64         <label class="control control--check" for="soundOn">
65             <span class="control__name">Choques</span>
66             <input type="checkbox" id="soundOn" checked>
67         </label>
68
69         <label class="control" for="volume">
70             <span class="control__name">Volumen</span>
71             <input type="range" id="volume" min="0" max="1" step="0.01" value="0.6">
72             <output class="control__value" id="volumeOut">0.60</output>
73         </label>
74     </div>
75
76     <button type="button" class="controls__reset" id="resetBtn">Reiniciar</button>
77 </aside>
78
79 <script type="module" src="./assets/js/main.js"></script>
80 </body>
81
82 </html>

```

4.2 assets/css/styles.css

Estilos del panel informativo y de la caja de controles: tarjeta translúcida con blur, sliders personalizados (pista degradada y thumb con glow), casilla y diseño responsive.

assets/css/styles.css

```

1  * {
2      box-sizing: border-box;
3  }
4
5  html,
6  body {
7      width: 100%;
8      height: 100%;
9      margin: 0;
10     overflow: hidden;
11 }
12
13 body {
14     font-family: Arial, sans-serif;
15     background: #07111f;
16 }
17
18 canvas {
19     display: block;
20 }
21
22 .panel {
23     position: fixed;
24     z-index: 1;
25     top: 20px;
26     left: 20px;
27     padding: 14px 18px;
28     color: white;
29     background: rgba(5, 18, 36, 0.75);
30     border-left: 4px solid #22d3ee;
31     border-radius: 8px;
32 }
33
34 .panel h1 {
35     margin: 0 0 5px;
36     font-size: 1.25rem;
37 }
38
39 .panel p {
40     margin: 0;
41     font-size: 0.9rem;

```

```

42 }
43
44 /* ===== Caja de controles ===== */
45 .controls {
46     position: fixed;
47     z-index: 2;
48     top: 20px;
49     right: 20px;
50     width: 260px;
51     padding: 18px;
52     color: #eaf6ff;
53     background: linear-gradient(160deg, rgba(9, 24, 44, 0.92), rgba(5, 15, 30, 0.92));
54     border: 1px solid rgba(34, 211, 238, 0.35);
55     border-radius: 14px;
56     box-shadow: 0 18px 40px rgba(0, 0, 0, 0.45), inset 0 1px 0 rgba(255, 255, 255, 0.06);
57     backdrop-filter: blur(6px);
58     max-height: calc(100vh - 40px);
59     overflow-y: auto;
60 }
61
62 .control-group + .control-group {
63     margin-top: 12px;
64 }
65
66 .controls h2 {
67     margin: 0 0 14px;
68     font-size: 1rem;
69     letter-spacing: 0.14em;
70     text-transform: uppercase;
71     color: #7de3f4;
72 }
73
74 .control-group {
75     padding: 12px;
76     border-radius: 10px;
77     background: rgba(255, 255, 255, 0.04);
78     border: 1px solid rgba(255, 255, 255, 0.06);
79 }
80
81 .control-group__title {
82     display: block;
83     margin-bottom: 10px;
84     font-size: 0.78rem;
85     text-transform: uppercase;
86     letter-spacing: 0.08em;
87     color: #9fb6cc;
88 }
89
90 .control {
91     display: grid;
92     grid-template-columns: 48px 1fr 44px;
93     align-items: center;
94     gap: 8px;
95     margin-bottom: 10px;
96 }
97
98 .control:last-child {
99     margin-bottom: 0;
100 }
101
102 .control--check {
103     grid-template-columns: 1fr auto;
104 }
105
106 .control--check input[type="checkbox"] {
107     width: 18px;
108     height: 18px;
109     accent-color: #22d3ee;
110     cursor: pointer;
111 }
112
113 .control__name {
114     font-size: 0.85rem;

```

```
115     font-weight: 700;
116     color: #cfe6f7;
117 }
118
119 .control__value {
120     font-size: 0.8rem;
121     font-variant-numeric: tabular-nums;
122     text-align: right;
123     color: #7de3f4;
124 }
125
126 .control input[type="range"] {
127     -webkit-appearance: none;
128     appearance: none;
129     width: 100%;
130     height: 6px;
131     border-radius: 999px;
132     background: linear-gradient(90deg, #22d3ee, #3b82f6);
133     outline: none;
134     cursor: pointer;
135 }
136
137 .control input[type="range"]::-webkit-slider-thumb {
138     -webkit-appearance: none;
139     appearance: none;
140     width: 16px;
141     height: 16px;
142     border-radius: 50%;
143     background: #eaf6ff;
144     border: 2px solid #22d3ee;
145     box-shadow: 0 0 8px rgba(34, 211, 238, 0.8);
146     transition: transform 0.1s ease;
147 }
148
149 .control input[type="range"]::-webkit-slider-thumb:hover {
150     transform: scale(1.15);
151 }
152
153 .control input[type="range"]::-moz-range-thumb {
154     width: 16px;
155     height: 16px;
156     border-radius: 50%;
157     background: #eaf6ff;
158     border: 2px solid #22d3ee;
159     box-shadow: 0 0 8px rgba(34, 211, 238, 0.8);
160 }
161
162 .controls__reset {
163     width: 100%;
164     margin-top: 14px;
165     padding: 9px 12px;
166     font-size: 0.85rem;
167     font-weight: 700;
168     letter-spacing: 0.05em;
169     color: #eaf6ff;
170     background: rgba(34, 211, 238, 0.15);
171     border: 1px solid rgba(34, 211, 238, 0.5);
172     border-radius: 9px;
173     cursor: pointer;
174     transition: background 0.15s ease;
175 }
176
177 .controls__reset:hover {
178     background: rgba(34, 211, 238, 0.3);
179 }
180
181 @media (max-width: 640px) {
182     .controls {
183         width: calc(100% - 40px);
184         top: auto;
185         bottom: 20px;
186     }
187 }
```

4.3 assets/js/main.js

Lógica completa: escena, cámara, iluminación, cubo de cristal, piso, esferas, controles, colisiones, chispas, sonido y post-proceso.

assets/js/main.js

```
1  import * as THREE from 'three';
2  import { OrbitControls } from 'three/addons/controls/OrbitControls.js';
3  import { RoundedBoxGeometry } from 'three/addons/geometries/RoundedBoxGeometry.js';
4  import { RoomEnvironment } from 'three/addons/environments/RoomEnvironment.js';
5  import { EffectComposer } from 'three/addons/postprocessing/EffectComposer.js';
6  import { RenderPass } from 'three/addons/postprocessing/RenderPass.js';
7  import { UnrealBloomPass } from 'three/addons/postprocessing/UnrealBloomPass.js';
8  import { OutputPass } from 'three/addons/postprocessing/OutputPass.js';
9
10 const scene = new THREE.Scene();
11 scene.background = new THREE.Color(0x05070c);
12 scene.fog = new THREE.Fog(0x05070c, 20, 58);
13
14 const camera = new THREE.PerspectiveCamera(
15     60,
16     window.innerWidth / window.innerHeight,
17     0.1,
18     100
19 );
20 camera.position.set(9, 7, 12);
21 camera.lookAt(0, 0, 0);
22
23 const renderer = new THREE.WebGLRenderer({ antialias: true });
24 renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2));
25 renderer.setSize(window.innerWidth, window.innerHeight);
26 renderer.shadowMap.enabled = true;
27 renderer.shadowMap.type = THREE.PCFSoftShadowMap;
28 renderer.toneMapping = THREE.ACESFilmicToneMapping;
29 renderer.toneMappingExposure = 0.95;
30 document.body.appendChild(renderer.domElement);
31
32 // Entorno HDR sintético para reflejos/refracciones realistas
33 const pmrem = new THREE.PMREMGGenerator(renderer);
34 scene.environment = pmrem.fromScene(new RoomEnvironment(), 0.04).texture;
35 scene.environmentIntensity = 0.45;
36
37 const controls = new OrbitControls(camera, renderer.domElement);
38 controls.enableDamping = true;
39 controls.dampingFactor = 0.05;
40 controls.minDistance = 5;
41 controls.maxDistance = 40;
42
43 scene.add(new THREE.AmbientLight(0xffffff, 1.1));
44 scene.add(new THREE.HemisphereLight(0x9fd8ff, 0x1a2436, 0.8));
45 const light = new THREE.DirectionalLight(0xffffff, 2.4);
46 light.position.set(6, 12, 7);
47 light.castShadow = true;
48 light.shadow.mapSize.set(2048, 2048);
49 light.shadow.camera.near = 1;
50 light.shadow.camera.far = 45;
51 light.shadow.camera.left = -12;
52 light.shadow.camera.right = 12;
53 light.shadow.camera.top = 12;
54 light.shadow.camera.bottom = -12;
55 light.shadow.bias = -0.0005;
56 scene.add(light);
57
58 const boxSize = 10;
59 const radius = 0.5;
60 const limit = boxSize / 2 - radius;
61 const half = boxSize / 2;
62
63 // Cubo de cristal sin bordes: aristas ligeramente redondeadas que
64 // atrapan la luz, transmisión total y doble refracción (grosor).
65 const boxGeometry = new RoundedBoxGeometry(boxSize, boxSize, boxSize, 6, 0.12);
```

```

66 const glassMaterial = new THREE.MeshPhysicalMaterial({
67     color: 0xffffffff,
68     metalness: 0,
69     roughness: 0.03,
70     transmission: 1,
71     ior: 1.48,
72     thickness: 0.35,
73     attenuationColor: new THREE.Color(0xcdeeff),
74     attenuationDistance: 3,
75     clearcoat: 1,
76     clearcoatRoughness: 0.06,
77     specularIntensity: 1,
78     envMapIntensity: 1.0,
79     side: THREE.DoubleSide,
80     transparent: true,
81     depthWrite: false
82 });
83 const glassBox = new THREE.Mesh(boxGeometry, glassMaterial);
84 glassBox.renderOrder = 2;
85 scene.add(glassBox);
86
87 // ===== Plano: piso de obsidiana pulida + rejilla + sombras =====
88 const floorY = -half;
89 const FLOOR_SIZE = 80;
90
91 // Acabado realista tipo obsidiana pulida: reflejo suave del entorno
92 // (sin espejo perfecto, para evitar destellos) y recibe las sombras.
93 const floor = new THREE.Mesh(
94     new THREE.PlaneGeometry(FLOOR_SIZE, FLOOR_SIZE),
95     new THREE.MeshStandardMaterial({
96         color: 0x05070c,
97         roughness: 0.45,
98         metalness: 0.0,
99         envMapIntensity: 0.25
100     })
101 );
102 floor.rotation.x = -Math.PI / 2;
103 floor.position.y = floorY - 0.02;
104 floor.receiveShadow = true;
105 scene.add(floor);
106
107 const grid = new THREE.GridHelper(60, 60, 0x2aa9cf, 0x123647);
108 grid.position.y = floorY - 0.012;
109 grid.material.transparent = true;
110 grid.material.opacity = 0.26;
111 grid.material.depthWrite = false;
112 scene.add(grid);
113
114 // Halo reactivo bajo el piso: pulsa con cada choque
115 const floorGlow = new THREE.Mesh(
116     new THREE.RingGeometry(half * 0.25, half * 1.05, 64),
117     new THREE.MeshBasicMaterial({
118         color: 0x1e9fd0,
119         transparent: true,
120         opacity: 0,
121         side: THREE.DoubleSide,
122         depthWrite: false,
123         blending: THREE.AdditiveBlending
124     })
125 );
126 floorGlow.rotation.x = -Math.PI / 2;
127 floorGlow.position.y = floorY - 0.008;
128 scene.add(floorGlow);
129 let glowPulse = 0;
130
131 function pulseFloor(amount) {
132     glowPulse = Math.min(glowPulse + amount, 1);
133 }
134
135 // ===== Esferas =====
136 const sphereGeometry = new THREE.SphereGeometry(radius, 32, 32);
137 const sphereColors = [
138     0xff7043, 0x42a5f5, 0x66bb6a, 0xffca28,

```

```

139     0xab47bc, 0xef5350, 0x26c6da, 0xd4e157
140 ];
141 const spheres = [];
142
143 const DEFAULT_BASE_SPEED = { x: 0.035, y: 0.027, z: 0.041 };
144 const baseSpeed = { ...DEFAULT_BASE_SPEED };
145
146 function randomSign() {
147     return Math.random() < 0.5 ? -1 : 1;
148 }
149
150 function addSphere() {
151     const mesh = new THREE.Mesh(
152         sphereGeometry,
153         new THREE.MeshPhysicalMaterial({
154             color: sphereColors[spheres.length % sphereColors.length],
155             roughness: 0.15,
156             metalness: 0,
157             clearcoat: 1,
158             clearcoatRoughness: 0.04,
159             envMapIntensity: 1.2
160         })
161     );
162     mesh.castShadow = true;
163     mesh.receiveShadow = true;
164     mesh.position.set(
165         THREE.MathUtils.randFloat(-limit, limit),
166         THREE.MathUtils.randFloat(-limit, limit),
167         THREE.MathUtils.randFloat(-limit, limit)
168     );
169     const velocity = new THREE.Vector3(
170         baseSpeed.x * randomSign(),
171         baseSpeed.y * randomSign(),
172         baseSpeed.z * randomSign()
173     );
174     scene.add(mesh);
175     spheres.push({ mesh, velocity });
176 }
177
178 function removeSphere() {
179     const s = spheres.pop();
180     if (!s) return;
181     scene.remove(s.mesh);
182     s.mesh.material.dispose();
183 }
184
185 function setSphereCount(n) {
186     while (spheres.length < n) addSphere();
187     while (spheres.length > n) removeSphere();
188 }
189
190 // Reescala la velocidad de todas las esferas conservando el sentido de cada eje
191 function applyBaseSpeed() {
192     for (const s of spheres) {
193         ['x', 'y', 'z'].forEach((axis) => {
194             const dir = s.velocity[axis] >= 0 ? 1 : -1;
195             s.velocity[axis] = dir * baseSpeed[axis];
196         });
197     }
198 }
199
200 // ===== Caja de controles =====
201 const countInput = document.getElementById('count');
202 const countOut = document.getElementById('countOut');
203 countInput.addEventListener('input', () => {
204     const n = parseInt(countInput.value, 10);
205     countOut.textContent = n;
206     setSphereCount(n);
207 });
208
209 const axisInputs = {
210     x: document.getElementById('velX'),
211     y: document.getElementById('velY'),

```

```

212     z: document.getElementById('velZ')
213 };
214 const axisOutputs = {
215     x: document.getElementById('velXOut'),
216     y: document.getElementById('velYOut'),
217     z: document.getElementById('velZOut')
218 };
219
220 function bindAxis(axis) {
221     const input = axisInputs[axis];
222     const output = axisOutputs[axis];
223     const apply = () => {
224         const value = Math.abs(parseFloat(input.value));
225         baseSpeed[axis] = value;
226         output.textContent = value.toFixed(3);
227         applyBaseSpeed();
228     };
229     input.addEventListener('input', apply);
230     output.textContent = Math.abs(parseFloat(input.value)).toFixed(3);
231 }
232 ['x', 'y', 'z'].forEach(bindAxis);
233
234 let soundEnabled = document.getElementById('soundOn').checked;
235 let masterVolume = parseFloat(document.getElementById('volume').value);
236 document.getElementById('soundOn').addEventListener('change', (e) => {
237     soundEnabled = e.target.checked;
238 });
239 const volumeInput = document.getElementById('volume');
240 const volumeOut = document.getElementById('volumeOut');
241 volumeInput.addEventListener('input', () => {
242     masterVolume = parseFloat(volumeInput.value);
243     volumeOut.textContent = masterVolume.toFixed(2);
244 });
245 volumeOut.textContent = masterVolume.toFixed(2);
246
247 document.getElementById('resetBtn').addEventListener('click', () => {
248     ['x', 'y', 'z'].forEach((axis) => {
249         axisInputs[axis].value = DEFAULT_BASE_SPEED[axis];
250         baseSpeed[axis] = DEFAULT_BASE_SPEED[axis];
251         axisOutputs[axis].textContent = DEFAULT_BASE_SPEED[axis].toFixed(3);
252     });
253     for (const s of spheres) {
254         s.mesh.position.set(
255             THREE.MathUtils.randFloat(-limit, limit),
256             THREE.MathUtils.randFloat(-limit, limit),
257             THREE.MathUtils.randFloat(-limit, limit)
258         );
259         s.velocity.set(
260             baseSpeed.x * randomSign(),
261             baseSpeed.y * randomSign(),
262             baseSpeed.z * randomSign()
263         );
264     }
265 });
266
267 // ===== Sonido de choque (Web Audio API) =====
268 let audioCtx = null;
269 let lastSoundAt = 0;
270
271 function ensureAudio() {
272     if (!audioCtx) {
273         const Ctx = window.AudioContext || window.webkitAudioContext;
274         if (Ctx) audioCtx = new Ctx();
275     }
276     if (audioCtx && audioCtx.state === 'suspended') audioCtx.resume();
277 }
278 window.addEventListener('pointerdown', ensureAudio);
279
280 function playClack(intensity) {
281     if (!soundEnabled || masterVolume <= 0) return;
282     ensureAudio();
283     if (!audioCtx) return;
284

```

```

285     const t = performance.now();
286     if (t - lastSoundAt < 25) return; // evita saturar con choques simultaneos
287     lastSoundAt = t;
288
289     const now = audioCtx.currentTime;
290     const length = Math.floor(audioCtx.sampleRate * 0.14);
291     const buffer = audioCtx.createBuffer(1, length, audioCtx.sampleRate);
292     const data = buffer.getChannelData(0);
293     for (let i = 0; i < length; i++) {
294         data[i] = (Math.random() * 2 - 1) * Math.pow(1 - i / length, 3);
295     }
296
297     const src = audioCtx.createBufferSource();
298     src.buffer = buffer;
299
300     const filter = audioCtx.createBiquadFilter();
301     filter.type = 'bandpass';
302     filter.frequency.value = 700 + intensity * 2600;
303     filter.Q.value = 1.1;
304
305     const gain = audioCtx.createGain();
306     gain.gain.value = Math.min(0.4, 0.08 + intensity * 0.5) * masterVolume;
307
308     src.connect(filter);
309     filter.connect(gain);
310     gain.connect(audioCtx.destination);
311     src.start(now);
312     src.stop(now + 0.14);
313 }
314
315 // ===== Marcas de impacto contra las caras =====
316 const markerDuration = 0.7;
317 const activeMarkers = [];
318 const markerGeometry = new THREE.RingGeometry(radius * 0.6, radius * 1.6, 32);
319
320 function spawnMarker(position, normal) {
321     const material = new THREE.MeshBasicMaterial({
322         color: 0xffff2b0,
323         transparent: true,
324         opacity: 0.9,
325         side: THREE.DoubleSide,
326         depthWrite: false
327     });
328     const marker = new THREE.Mesh(markerGeometry, material);
329     marker.position.copy(position).addScaledVector(normal, 0.01);
330     marker.lookAt(position.clone().add(normal));
331     scene.add(marker);
332     activeMarkers.push({ mesh: marker, material, life: markerDuration });
333 }
334
335 function updateMarkers(delta) {
336     for (let i = activeMarkers.length - 1; i >= 0; i--) {
337         const m = activeMarkers[i];
338         m.life -= delta;
339         const t = Math.max(m.life / markerDuration, 0);
340         m.material.opacity = 0.9 * t;
341         m.mesh.scale.setScalar(1 + (1 - t) * 1.5);
342         if (m.life <= 0) {
343             scene.remove(m.mesh);
344             m.material.dispose();
345             activeMarkers.splice(i, 1);
346         }
347     }
348 }
349
350 // ===== Chispas =====
351 const MAX_BURSTS = 40;
352 const sparkBursts = [];
353
354 function spawnSparks(position, intensity, tint) {
355     if (sparkBursts.length >= MAX_BURSTS) return;
356
357     const count = 8 + Math.round(intensity * 10);

```

```

358     const positions = new Float32Array(count * 3);
359     const vels = [];
360     for (let i = 0; i < count; i++) {
361         positions[i * 3] = position.x;
362         positions[i * 3 + 1] = position.y;
363         positions[i * 3 + 2] = position.z;
364         vels.push(
365             new THREE.Vector3(
366                 Math.random() * 2 - 1,
367                 Math.random() * 2 - 1,
368                 Math.random() * 2 - 1
369             ).normalize().multiplyScalar(THREE.MathUtils.randFloat(1.5, 4) * (0.6 + intensity))
370         );
371     }
372
373     const geo = new THREE.BufferGeometry();
374     geo.setAttribute('position', new THREE.BufferAttribute(positions, 3));
375     const mat = new THREE.PointsMaterial({
376         color: tint,
377         size: 0.14,
378         transparent: true,
379         opacity: 1,
380         depthWrite: false,
381         blending: THREE.AdditiveBlending
382     });
383     const points = new THREE.Points(geo, mat);
384     scene.add(points);
385     sparkBursts.push({ points, geo, mat, vels, life: 0.45, maxLife: 0.45 });
386 }
387
388 function updateSparks(delta) {
389     for (let i = sparkBursts.length - 1; i >= 0; i--) {
390         const b = sparkBursts[i];
391         b.life -= delta;
392         const arr = b.geo.attributes.position.array;
393         for (let j = 0; j < b.vels.length; j++) {
394             arr[j * 3] += b.vels[j].x * delta;
395             arr[j * 3 + 1] += b.vels[j].y * delta;
396             arr[j * 3 + 2] += b.vels[j].z * delta;
397             b.vels[j].multiplyScalar(0.9);
398         }
399         b.geo.attributes.position.needsUpdate = true;
400         b.mat.opacity = Math.max(b.life / b.maxLife, 0);
401         if (b.life <= 0) {
402             scene.remove(b.points);
403             b.geo.dispose();
404             b.mat.dispose();
405             sparkBursts.splice(i, 1);
406         }
407     }
408 }
409
410 // ===== Colisiones =====
411 const AXES = ['x', 'y', 'z'];
412 const tmpNormal = new THREE.Vector3();
413 const tmpRel = new THREE.Vector3();
414
415 function collideWithWalls(s) {
416     for (const axis of AXES) {
417         const p = s.mesh.position[axis];
418         if (p >= limit || p <= -limit) {
419             s.velocity[axis] *= -1;
420             s.mesh.position[axis] = THREE.MathUtils.clamp(p, -limit, limit);
421
422             const contact = s.mesh.position.clone();
423             contact[axis] = Math.sign(p) * half;
424             const normal = new THREE.Vector3();
425             normal[axis] = -Math.sign(p);
426
427             const speed = Math.abs(s.velocity[axis]);
428             const intensity = THREE.MathUtils.clamp(speed / 0.12, 0.1, 1);
429             spawnMarker(contact, normal);
430             spawnSparks(contact, intensity, 0xffff2b0);

```

```

431         playClack(intensity);
432         pulseFloor(intensity * 0.6);
433     }
434 }
435 }
436
437 function collidePair(a, b) {
438     tmpNormal.subVectors(b.mesh.position, a.mesh.position);
439     const dist = tmpNormal.length();
440     const minDist = radius * 2;
441     if (dist === 0 || dist >= minDist) return;
442
443     tmpNormal.multiplyScalar(1 / dist); // normal a -> b
444
445     // separar el solapamiento
446     const overlap = (minDist - dist) / 2;
447     a.mesh.position.addScaledVector(tmpNormal, -overlap);
448     b.mesh.position.addScaledVector(tmpNormal, overlap);
449
450     // choque elastico de masas iguales: se intercambia la componente normal
451     tmpRel.subVectors(a.velocity, b.velocity);
452     const sep = tmpRel.dot(tmpNormal);
453     if (sep <= 0) return; // ya se estan separando
454
455     a.velocity.addScaledVector(tmpNormal, -sep);
456     b.velocity.addScaledVector(tmpNormal, sep);
457
458     const contact = a.mesh.position.clone().addScaledVector(tmpNormal, radius);
459     const intensity = THREE.MathUtils.clamp(sep / 0.15, 0.15, 1);
460     spawnSparks(contact, intensity, 0xbdecff);
461     playClack(intensity);
462     pulseFloor(intensity * 0.4);
463 }
464
465 // ===== Post-proceso: bloom cinematografico =====
466 const composer = new EffectComposer(renderer);
467 composer.addPass(new RenderPass(scene, camera));
468 const bloomPass = new UnrealBloomPass(
469     new THREE.Vector2(window.innerWidth, window.innerHeight),
470     0.32, // strength
471     0.5, // radius
472     0.86 // threshold
473 );
474 composer.addPass(bloomPass);
475 composer.addPass(new OutputPass());
476
477 // ===== Bucle =====
478 let lastTime = performance.now();
479
480 function animate() {
481     const now = performance.now();
482     const delta = Math.min((now - lastTime) / 1000, 0.05);
483     lastTime = now;
484
485     for (const s of spheres) {
486         s.mesh.position.add(s.velocity);
487         collideWithWalls(s);
488     }
489
490     for (let i = 0; i < spheres.length; i++) {
491         for (let j = i + 1; j < spheres.length; j++) {
492             collidePair(spheres[i], spheres[j]);
493         }
494     }
495
496     updateMarkers(delta);
497     updateSparks(delta);
498
499     // halo del piso: decae suave y respira ligeramente
500     glowPulse = Math.max(glowPulse - delta * 1.8, 0);
501     const breathe = 0.015 + 0.01 * Math.sin(now * 0.002);
502     floorGlow.material.opacity = breathe + glowPulse * 0.4;
503     floorGlow.scale.setScalar(1 + glowPulse * 0.15);

```

```
504     grid.material.opacity = 0.28 + glowPulse * 0.3;
505
506     controls.update();
507     composer.render();
508 }
509
510 setSphereCount(parseInt(countInput.value, 10));
511 renderer.setAnimationLoop(animate);
512
513 window.addEventListener('resize', () => {
514     camera.aspect = window.innerWidth / window.innerHeight;
515     camera.updateProjectionMatrix();
516     renderer.setSize(window.innerWidth, window.innerHeight);
517     composer.setSize(window.innerWidth, window.innerHeight);
518     bloomPass.resolution.set(window.innerWidth, window.innerHeight);
519 });
520
```